

Training a Deep Convolutional GAN (DCGAN) on the CelebA Dataset

Chirag Khandelwal

1 Objective

The goal of this project is to train a **Deep Convolutional Generative Adversarial Network (DCGAN)** on the CelebA dataset in order to generate realistic, synthetic human face images. A GAN works by pitting two neural networks against each other in a competitive game: one network learns to create fake images, while the other learns to tell real images apart from fake ones. Through this competition, the model gradually learns to produce face images that look increasingly realistic.

2 Dataset

This project uses the **CelebA (CelebFaces Attributes) dataset**, a large-scale dataset containing more than **200,000 celebrity face images**, each annotated with various facial attributes.

For this project:

- All images were resized to 64×64 pixels to reduce computational cost and keep training times manageable.
- Pixel values were normalized to the range $[-1, 1]$ so that they align with the output range of the Generator's final **Tanh** activation.

3 Architecture

A DCGAN consists of two competing neural networks: a **Generator** and a **Discriminator**. They are trained together, with each one trying to "outsmart" the other.

3.1 Generator: The Artist

The Generator's job is to act like an **artist** – it starts with random noise (a vector of random numbers) and learns to transform that noise into a realistic-looking face image. Think of it as sculpting a detailed face out of a block of random static.

Key components:

- **ConvTranspose2D (Transposed Convolution):** Used for *upsampling*. It takes a small input (like the random noise vector) and gradually expands it into a larger image, layer by layer, similar to "unzipping" compressed information back into a full picture.

- **BatchNorm (Batch Normalization):** Helps stabilize and speed up training by normalizing the outputs of each layer, preventing the network from producing wildly inconsistent values.
- **ReLU (Rectified Linear Unit):** An activation function used in the hidden layers that helps the network learn complex, non-linear patterns.
- **Tanh:** Used in the final output layer to squash pixel values into the range $[-1, 1]$, matching the normalized range of the real training images.

3.2 Discriminator: The Detective

The Discriminator acts like a **detective** – its job is to look at an image and decide whether it is a real photo from the CelebA dataset or a fake one created by the Generator. As training progresses, it becomes better and better at spotting fakes, which in turn forces the Generator to improve.

Key components:

- **Conv2D (Convolution):** Used for *downsampling*. It scans the input image and gradually compresses it into a smaller set of meaningful features, similar to summarizing a picture into its key visual clues.
- **BatchNorm (Batch Normalization):** Just like in the Generator, this stabilizes training by normalizing intermediate outputs.
- **LeakyReLU:** An activation function similar to ReLU, but it allows a small, non-zero gradient for negative values. This helps prevent "dead neurons" and keeps gradients flowing during training.
- **Sigmoid:** Used in the final layer to output a single probability value between 0 and 1, representing how confident the Discriminator is that the input image is real (closer to 1) versus fake (closer to 0).

4 Hyperparameters

The table below summarizes the key hyperparameters used to train the DCGAN model.

Table 1: Training Hyperparameters

Hyperparameter	Value
Batch Size	128
Learning Rate	0.0002
Latent Dimension	100
Optimizer	Adam
Epochs	25–50

5 Results

5.1 Generated Face Samples



Figure 1: Sample face images generated by the DCGAN after training.

5.2 Generator and Discriminator Loss Curve

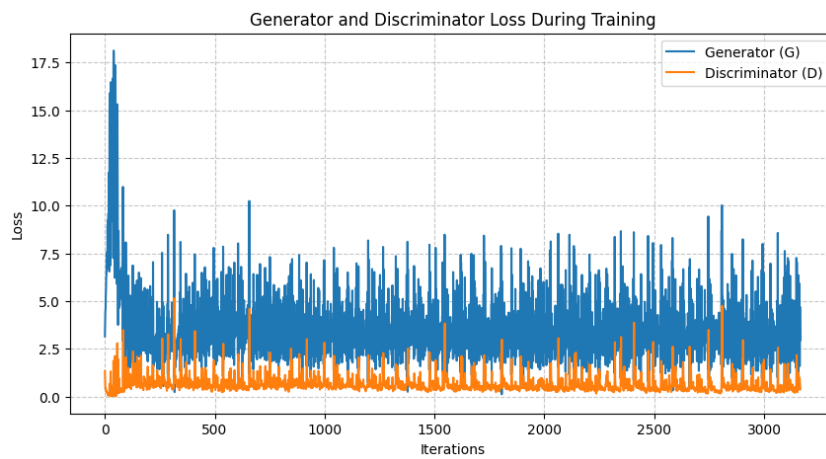


Figure 2: Generator loss vs. Discriminator loss over training epochs.

6 Observations

During the course of training, the following trends were observed:

- The **Generator gradually learned facial structures**, starting from random noise and progressively forming recognizable features such as eyes, noses, and facial outlines.
- The **Discriminator became harder to fool over time**, as it learned increasingly subtle cues to distinguish real images from generated ones.
- Training **occasionally showed instability**, with loss values fluctuating rather than steadily decreasing – a common and expected characteristic of adversarial training (GANs).
- **Generated faces became visibly sharper** as training progressed through later epochs, with finer details and more realistic textures compared to early, blurry outputs.